دستگاه‌های سخت‌افزاری دیگری با مدارهای مجتمع خاص کاربردی[1] (ASIC) هستند که وظایف خود را بدون داشتن یک سیستم‌عامل انجام می‌دهند.

استفاده از سیستم‌های تعبیه‌شده رو به گسترش است. قدرت این دستگاه‌ها نیز توأمان به صورت واحدی مستقل و به عنوان عضوی از شبکه‌ها و وب در حال افزایش است. حتی در حال حاضر، کل منازل می‌توانند کامپیوتری شوند، لذا تنها کافی است که یک کامپیوتر مرکزی—یک کامپیوتر همه‌منظوره و یا یک سیستم تعبیه‌شده—گرمایش و روشنایی، سیستم‌های هشدار و حتی قهوه‌سازها را کنترل کند. مالک خانه با دسترسی وب می‌تواند به منزل بگوید که پیش از رسیدن او به خانه، گرم شود. روزی می‌رسد که یخچال قادر خواهد بود به خواربارفروش محل زنگ زده و خبر دهد که شیر درون یخچال، ته کشیده است.

سیستم‌های تعبیه‌شده تقریباً همواره **سیستم‌عامل‌های بلادرنگ**[2] را اجرا می‌کنند. سیستم بلادرنگ زمانی استفاده می‌شود که نیازهای زمانی سفت و سختی روی عملیات پردازنده یا گردش داده‌ها وجود دارد؛ بنابراین، اغلب به عنوان یک دستگاه کنترل در کاربردهای اختصاصی به کار می‌رود. حسگرها داده‌ها را به کامپیوتر می‌آورند. کامپیوتر بایستی داده‌ها را تحلیل کند و احتمالاً کنترل‌ها را تنظیم کند تا ورودی‌های حسگر اصلاح شوند. سیستم‌هایی که آزمایش‌های علمی، سیستم‌های تصویربرداری پزشکی، سیستم‌های کنترل صنعتی و سیستم‌های نمایش خاص را کنترل می‌کنند، سیستم‌های بلادرنگ هستند. برخی سیستم‌های تزریق سوخت موتور خودروها، کنترلرهای وسایل خانگی و سیستم‌های تسلیحاتی نیز جزء سیستم‌های بلادرنگ به حساب می‌آیند.

یک سیستم بلادرنگ، محدویت‌های زمانی ثابت خوش‌تعریفی دارد. بایستی پردازش در محدودیت‌های زمانی تعریف‌شده انجام شود، یا سیستم از کار خواهد افتاد. برای نمونه، اگر یک روبات در داخل خودرویی که می‌سازد خراب شود، دیگر نمی‌توان بازوی روبات را با هدایت متوقف کرد. سیستم بلادرنگ فقط در صورتی به درستی کار می‌کند که نتایج درستی را با محدویت‌های زمانی خودش برگرداند. این سیستم را با یک سیستم لپ‌تاپ معمول مقایسه کنید که در آن داشتن زمان پاسخ سریع مطلوب است (اما حیاتی نیست).

در فصل ۵، امکانات زمانبندی مورد نیاز پیاده‌سازی کارکرد بلادرنگ را در یک سیستم‌عامل بررسی می‌کنیم، و در فصل ۲۰، مؤلفه‌های بلادرنگ لینوکس را مطرح می‌کنیم.

۱-۱۱ سیستم‌عامل‌های رایگان و متن‌باز

مطالعه‌ی سیستم‌عامل‌ها با دسترس‌پذیری نسخه‌های بی‌شمار رایگان و متن‌باز نرم‌افزار آسان‌تر شده است. هر دوی **سیستم‌عامل‌های رایگان و سیستم‌عامل‌های متن‌باز**[3] به‌جای کد دودویی کامپایل‌شده، در قالب کد اصلی در دسترس قرار دارند. هرچند توجه داشته باشید که نرم‌افزار رایگان و نرم‌افزار متن‌باز دو موضوع متفاوتی هستند که توسط گروه‌های متفاوتی از

1. Application-specific integrated circuits - ASICs
2. Real-time operating systems
3. Open source operating systems

افراد حمایت می‌شوند (برای بحث راجع به این موضوع سایت http://gnu.org/philosophy/open-source-misses-the-point.html/ را ببینید). نرم‌افزار رایگان (بعضاً تحت عنوان free/libre software به آن رجوع می‌شود) نه تنها کد اصلی را در دسترس قرار می‌دهد بلکه مجوز می‌دهد که بدون هیچ هزینه‌ای، استفاده، بازتوزیع و اصلاح شود. نرم‌افزار متن‌باز لزوماً چنین مجوزی را صادر نمی‌کند. لذا، هرچند که تمامی نرم‌افزارهای رایگان متن‌باز هستند، برخی نرم‌افزارهای متن‌باز «رایگان» نیستند. GNU/Linux معروف‌ترین سیستم‌عامل متن‌باز با برخی توزیع رایگان و بقیه صرفاً متن‌باز است (http://www.gnu.org/distros/). مایکروسافت ویندوز مثال بسیار آشنایی از رویکرد **متن‌بسته**[1] می‌باشد. ویندوز، نرم‌افزاری **خصوصی**[2] است—مایکروسافت مالک آن است، استفاده‌اش را محدود می‌کند، و به‌دقت از کد اصلی آن محافظت می‌کند. سیستم‌عمل macOS شرکت اپل، رویکردی ترکیبی بر می‌دارد. آن شامل یک هسته‌ی متن‌باز به نام Darwin است اما در عین حال، دارای اجزای متن‌بسته‌ی اختصاصی نیز می‌باشد.

کار بر روی کد اصلی به برنامه‌نویس این اجازه را می‌دهد که کد دودویی تولید کند تا بتواند روی یک سیستم اجرا شود. انجام عکس این کار – **مهندسی معکوس**[3]، یعنی استخراج کد اصلی از شکل دودویی—نیازمند کار زیاد بوده و امکان بازیابی موارد مفیدی چون توضیحات کد اصلی را میسر نمی‌سازد. یادگیری سیستم‌عامل با آزمایش کد اصلی، منافع دیگری نیز دارد. دانش‌جو با در دسترس داشتن کد اصلی می‌تواند سیستم‌عامل را تغییر داده و سپس کامپایل و کد را اجرا نماید تا حاصل تغییرات خود را تجربه کند که یک ابزار یادگیری عالی است. در این کتاب پروژه‌هایی لحاظ شده است که کد اصلی سیستم‌عامل را تغییر می‌دهند، ضمن این که الگوریتم‌ها در سطح بالایی تشریح می‌شوند تا مطمئن شویم تمامی مباحث مهم سیستم‌عامل پوشانده می‌شوند. در کل کتاب، اشاره‌هایی برای نمونه‌های متن‌باز، جهت مطالعه‌ی عمیق‌تر ارائه می‌دهیم.

منافع زیادی در سیستم‌عامل‌های متن‌باز وجود دارد که از آن جمله می‌توان به یک گروه از برنامه‌نویسان علاقه‌مند (و معمولاً، با کار بی‌مزد) اشاره کرد که به نوشتن، عیب‌یابی، تحلیل، پشتیبانی و تغییرات کد کمک می‌کنند. قابل استدلال است که کد اصلی باز، ایمن‌تر از کد اصلی بسته می‌باشد چرا که چشمان زیادی هستند که اندرون آن را می‌بینند. کد اصلی باز یقیناً عیب‌هایی خواهد داشت، اما متن باز ما را به این باور می‌رساند که عیب‌ها پیدا شده و با جدیت تعداد بی‌شماری از استفاده‌کنندگان و بینندگان کد، سریع‌تر ترمیم خواهند یافت. شرکت‌هایی که از فروش برنامه‌هایشان درآمد دارند به نظر مردد هستند که کد خود را باز نمایند، اما *RedHat* و شرکت‌های بی‌شمار دیگری هستند که این کار را کرده و نشان داده‌اند که به جای این که از بازبودن کد خود ضرری ببینند، شرکت‌های تجاری موفقی هم بوده‌اند. برای مثال، درآمد می‌تواند از طریق قراردادهای پشتیبانی و فروش سخت‌افزاری که نرم‌افزار روی آن اجرا می‌شود، حاصل شود.

1. Closed source
2. Proprietary
3. Reverse engineering

۱-۱۱-۱ تاریخچه

در روزهای آغازین محاسبات مدرن (یعنی، دهه‌ی ۱۹۵۰)، نرم‌افزار غالباً در قالب متن‌باز در دسترس بود. مهاجمان اولیه (مشتاقان کامپیوتر) در Tech Model Railroad Club دانشگاه MIT، برنامه‌های خودشان را در کشوها برای کار سایرین بر روی آنها رها می‌کردند. گروه‌های کاربری «Homebrew» در طی نشست‌های خود کد مبادله می‌کردند. گروه‌های کاربری شرکتی خاص، همچون شرکت تجهیزات دیجیتال[1] (DEC)، تشریک مساعی برنامه‌های متن‌باز را پذیرفته، روی نوارهایی گرد آورده و نوارها را به اعضای علاقه‌مند توزیع کردند. در سال ۱۹۷۰، سیستم‌عامل‌های Digital به‌صورت کد اصلی، بدون هیچ محدودیتی یا هشدار حق کپی توزیع شدند.

سرانجام، شرکت‌های کامپیوتری و نرم‌افزاری بر آن شدند تا استفاده از نرم‌افزار خودشان را به کامپیوترهای مجاز محدود کرده و از مشتری‌های خود پول بگیرند. فقط توزیع فایل‌های دودویی کامپایل‌شده از کد اصلی به جای خود کد اصلی به آنها کمک کرد تا در کنار حفاظت از کد و ایده‌های خودشان در مقابل رقبایشان به این هدف هم برسند. هرچند که گروه‌های کاربری Homebrew دهه‌ی ۱۹۷۰ در طی نشست‌های خود به‌مبادله‌ی کد می‌پرداختند، سیستم‌عامل‌های ماشین‌های سرگرمی (همچون CPM) خصوصی بودند. تا سال ۱۹۸۰، نرم‌افزار خصوصی مورد معمول بود.

۱-۱۱-۲ سیستم‌عامل‌های رایگان

برای مقابله با حرکت اِعمال محدودیت استفاده از نرم‌افزار و توزیع مجدد آن، Richard Stallman، توسعه‌ی یک سیستم‌عامل رایگان و سازگار با یونیکس به‌نام GNU (که اختصار عبارت بازگشتی «GNU's Not Unix» می‌باشد) را در سال ۱۹۸۴ شروع کرد. برای Stallman، «free » به آزادی در استفاده مربوط می‌شد نه قیمت. جنبش نرم‌افزار رایگان به‌دنبال این نیست که برای کپی، یک میزان پولی دریافت کند بلکه هدف آن رسیدن به چهار آزادی ویژه است: (۱) اجرای آزادانه‌ی برنامه، (۲) مطالعه و تغییر کد اصلی، و همین‌طور، دادن کپی یا فروش (۳) با تغییرات و یا (۴) بدون تغییرات. در سال ۱۹۸۵، Stallman بیانیه‌ی گنو را منتشر کرد که استدلال آن این بود که بایستی تمامی نرم‌افزارها رایگان باشند. او همچنین بنیاد نرم‌افزار رایگان[2] (FSF) را با هدف تشویق استفاده و نوشتن نرم‌افزار رایگان تشکیل داد.

FSF از حق کپی روی برنامه‌هایش استفاده می‌کند تا «کپی‌باقی[3]» را پیاده کند که شکلی از مجوزدهی است که Stallman اختراع کرد. کپی‌باقی‌کردن یک کار به هر کس که مالکیت یک کپی از کار را دارد چهار آزادی اساسی را می‌دهد که کار را رایگان کند با این شرط که توزیع مجدد بایستی این آزادی‌ها را حفظ کند. مجوز عمومی سراسری[4] (GPL) گنو، یک مجوز رایج است که

1. Digital Equipment Corporation
2. Free Software Foundation
3. Copyleft
4. General Public License

نرم‌افزار رایگان، تحت آن عرضه می‌شود. اساساً، GPL لازم دارد که کد اصلی با کل کدهای دودویی، توزیع شده و تمامی کپی‌ها (اعم از نسخه‌های اصلاح‌شده) با همان مجوز GPL عرضه شوند. مجوز "Creative Commons "Attribution Sharealike نیز یک مجوز کپی‌باقی است؛ «sharealike» شیوه‌ی دیگری از بیان ایده‌ی کپی‌باقی است.

۱-۱۱-۳ گنو/لینوکس

به عنوان مثالی از یک سیستم‌عامل متن‌باز، **گنو/لینوکس** را در نظر بگیرید. تا سال ۱۹۹۱، سیستم‌عامل گنو تقریباً تکمیل شد. پروژه‌ی گنو، کامپایلر، ادیتور، برنامه‌های سودمند، کتابخانه، و بازی توسعه داد—یعنی، هر چیزی نتوانست جای دیگری پیدا کند. با این وجود، هسته‌ی گنو هرگز برای اولین بار مهیا نشد. در سال ۱۹۹۱، یک دانشجو در فنلاند، به نام Linus Torvalds یک هسته با پایه‌ای مشابه یونیکس با استفاده از کامپایلرها و ابزارهای گنو عرضه کرد و از تمامی دنیا دعوت به همکاری کرد. ظهور اینترنت به این معنی بود که هرکس که علاقه‌مند باشد می‌تواند کد اصلی را برداشته، تغییر داده و به Torvalds ارسال کند. عرضه‌ی به‌روزرسانی‌های هفتگی این سیستم‌عامل که «لینوکس» نامیده می‌شد، امکان رشد سریعی به آن داد و توسط هزاران برنامه‌نویس بهبود پیدا کرد. در سال ۱۹۹۱، لینوکس نرم‌افزار رایگان نبود، چرا که مجوز آن تنها توزیع مجدد غیرتجاری را اجازه می‌داد. با این همه، در سال ۱۹۹۲، لینوکس را تحت GPL بازنشر داد که آن را نرم‌افزار رایگان ساخت (و نیز اصطلاحی که بعدها «متن‌باز» نام گرفت).

سیستم‌عامل نتیجه، گنو/لینوکس (با هسته‌ای که به‌درستی لینوکس نامیده شد اما سیستم‌عامل کاملی که شامل ابزار گنو می‌باشد گنو/لینوکس نامیده شد) صدها **توزیع** یکتا یا ساخت سفارشی از آن سیستم به راه انداخت. توزیع‌های اصلی عبارتند از: Red Hat، SUSE، Fedora، Debian، Slackware و Ubuntu. تنوع این توزیع‌ها به کارکرد، برنامه‌های سودمند، برنامه‌های کاربردی نصب‌شده، پشتیبانی سخت‌افزاری، رابط کاربر و هدف آنها بر می‌گردد. برای مثال، Red Hat Enterprise Linux برای استفاده‌ی تجاری گسترده عرضه می‌شود. PCLinuxOS یک live CD است—سیستم‌عاملی که می‌تواند از طریق یک CD-ROM بدون آنکه روی دیسک بوت سیستم نصب شود راه‌اندازی و اجرا شود. یک گونه از PCLinuxOS–به نام « PCLinuxOS Supergamer DVD» است—یک LiveDVD می‌باشد که شامل درایورهای گرافیکی و بازی‌هاست. یک gamer می‌تواند آن را روی هر سیستم سازگار به سادگی با راه‌اندازی از DVD اجرا کند. زمانی که gamer پایان می‌یابد، سیستم با یک بار راه‌اندازی مجدد به سیستم‌عامل نصب شده‌اش بر می‌گردد.

شما می‌توانید لینوکس را با استفاده از رویکرد ساده و رایگان زیر بر روی سیستم ویندوز (یا سیستم دیگر) اجرا کنید:

۱-ابزار رایگان Virtualbox VMM را از آدرس زیر بردارید و روی سیستم خودتان نصب کنید:

http://www.virtualbox.org/

۲- نصب کامل یک سیستم‌عامل را بر اساس یک تصویر نصب مثل یک CD، یا تصاویر از پیش‌ساخته‌ی سیستم‌عامل‌ها را انتخاب کنید که می‌توانند نصب شده و سریع‌تر اجرا شوند از سایتی مثل آدرس زیر،

http://virtualboxes.org/images/

این تصاویر با سیستم‌عامل‌ها و برنامه‌هایشان از پیش نصب شده هستند و گونه‌های زیادی از گنو/لینوکس را شامل هستند.

۳- ماشین مجازی را در Virtualbox بوت کنید.

یک روش جایگزین در استفاده از Virtualbox استفاده از برنامه‌ی رایگان Qemu (http://wiki.gemu.org/Download/) می‌باشد که شامل فرمان gemu –img برای تبدیل تصاویر Virtualbox به تصاویر Qemu به‌منظور import راحت آن‌هاست.

به همراه این کتاب، تصویری از ماشین مجازی گنو/لینوکس که نسخه‌ی Ubuntu را اجرا می‌کند تدارک دیده‌ایم. این تصویر، شامل کد اصلی گنو/لینوکس، به همراه ابزار توسعه‌ی نرم‌افزار است. در کل کتاب، مثال‌هایی مشمول تصویر گنو/لینوکس مزبور، به همراه یک کاربرد عملی با جزئیات در فصل ۲۰ پوشش می‌دهیم.

۱-۱۱-۴ یونیکس BSD

یونیکس BSD سابقه‌ی طولانی و پیچیده‌تری از لینوکس دارد که در سال ۱۹۷۸، به عنوان مشتقی از یونیکس AT&T آغاز شد. نسخه‌هایی از دانشگاه برکلی کالیفورنیا به شکل کد اصلی و دودویی عرضه شدند، اما متن‌باز نبودند چرا که مجوز AT&T را لازم داشتند. توسعه‌ی یونیکس BSD به‌واسطه‌ی شکایتی که AT&T داشت کندتر شد، اما سرانجام نسخه‌ی متن‌بازی که کاملاً قابل استفاده بود به نام 4.4BSD-lite در سال ۱۹۹۴ عرضه شد.

درست مانند لینوکس، یونیکس BSD هم دارای توزیع‌های زیادی است که از آن جمله می‌توان به FreeBSD، NetBSD، OpenBSD و DragonflyBSD اشاره کرد. برای پیمایش کد اصلی FreeBSD، تصویر ماشین مجازی نسخه‌ی مطلوب آن را به سادگی از اینترنت برداشته و در Virtualbox، چنان که در بالا برای لینوکس گفته شد، راه‌اندازی کنید. کد اصلی همراه توزیع می‌باشد و در مسیر /usr/src/ ذخیره می‌شود. کد اصلی هسته در /usr/src/sys قرار دارد. برای مثال، برای بررسی کد پیاده‌ساز حافظه‌ی مجازی در هسته‌ی FreeBSD، فایل‌های داخل /usr/src/sys/vm را ببینید. به روشی دیگر، می‌توانید کد اصلی را به‌سادگی به‌صورت برخط در آدرس https://svnweb.freebsd.org ببینید.

مثل بسیاری از پروژه‌های متن‌باز، این کد اصلی در یک **سیستم کنترل نسخه** قرار گرفته و کنترل می‌شود—در اینجا، «subversion» (https://subversion.apache.org/source-code). سیستم‌های کنترل نسخه به یک کاربر امکان می‌دهند تا درخت یک کد اصلی را به‌صورت کامل به کامپیوتر خودش منتقل کند و هرگونه تغییری که خواست بر روی نسخه‌اش انجام دهد تا آن‌ها را دیگران بتوانند بردارند. این سیستم‌ها امکانات دیگری نیز دارند، شامل تاریخچه‌ی کاملی از هر فایل و یک امکان شفاف‌سازی برخورد، در صورتی که فایل یکسانی

به‌صورت همروند تغییر داده شود. سیستم دیگر کنترل نسخه **git** می‌باشد که برای گنو/لینوکس، همین‌طور برنامه‌های دیگر استفاده می‌شود.

Darwin، مؤلفه‌ی اصلی هسته‌ی macOS، مبتنی بر یونیکس BSD بوده و متن‌باز هم هست. کد اصلی آن از آدرس http://www.opensource.apple.com/ قابل دسترسی است. هر یک از نسخه‌های macOS، اجزای متن‌باز خود را در این سایت عرضه می‌کنند. نام بسته‌ای که هسته را در بر دارد با «xnu» شروع می‌شود. همین‌طور، اپل ابزار برنامه‌نویسی، مستندسازی و پشتیبان وسیعی در http://developer.apple.com تدارک دیده است.

۱-۱۱-۵ سولاریس

سولاریس، سیستم‌عامل تجاری مبتی بر یونیکس شرکت سان میکروسیستمز[1] است. در آغاز، سیستم‌عامل سان، یعنی SunOS مبتنی بر یونیکس BSD بود. سان از سال ۱۹۹۱ به عنوان اساس سیستم‌عامل خود به سمت یونیکس AT&T System V حرکت کرد. در سال ۲۰۰۵، سان عمده‌ی کد سولاریس را به صورت پروژه‌ی OpenSolaris متن‌باز کرد. با این وجود، با خرید سان توسط اوراکل در سال ۲۰۰۹، وضعیت این پروژه به صورت مبهم باقی بماند.

گروه‌های متعدد علاقه‌مند به استفاده از OpenSolaris، امکانات آن را بسط دادند، و مجموعه‌ی کاری آنها Project Illumos است که از پایه‌ی OpenSolaris توسعه داده شده تا امکانات بیشتری را در بر گرفته و اساس محصولات متعدد شود. Illumos از آدرس http://wiki.illumos.org در دسترس می‌باشد.

۱-۱۱-۶ سیستم‌های متن‌باز به‌عنوان ابزار یادگیری

جنبش نرم‌افزار رایگان به سمت جذب برنامه‌نویسان حرفه‌ای است تا هزاران پروژه‌ی متن‌باز، از جمله سیستم‌عامل‌ها را ایجاد نمایند. سایت‌هایی چون http://freshmeat.net/ و http://distrowatch.com/، پورتال‌هایی برای بسیاری از این پروژه‌ها دارند. همان‌طور که پیشتر عنوان کردیم، پروژه‌های متن‌باز دانش‌جویان را قادر می‌سازد تا از کد اصلی به عنوان ابزار یادگیری استفاده نمایند. آنها می‌توانند برنامه‌ها را تغییر داده و آزمایش کنند، در یافتن و ترمیم عیب‌ها کمک کنند و در کنار آنها بتوانند سیستم‌عامل‌ها را با امکانات کامل، کامپایلرها، ابزار، رابط‌های کاربری و سایر انواع برنامه‌ها پیمایش کنند. دسترس‌پذیری کد اصلی پروژه‌های تاریخی همچون مالتیکس[2] می‌تواند به دانش‌جویان کمک کند تا با درک آن پروژه‌ها و دانش‌افزایی، قادر به کمک در پیاده‌سازی پروژه‌های جدیدی باشند.

حسن دیگر کار با سیستم‌عامل‌های متن‌باز، تنوع آن‌هاست. برای نمونه، گنو/لینوکس و یونیکس BSD هر دو سیستم‌عامل‌های متن‌باز هستند، اما هر یک از آنها اهداف، سودمندی و مجوز خاص خود را دارند. گاهاً مجوزها انحصاری نبوده و هم‌افزایی فکری روی می‌دهد که امکان پیشرفت‌های سریع پروژه‌های سیستم‌عامل را فراهم می‌کند. برای مثال، چندین مؤلفه‌ی

1. Sun Microsystems
2. MULTICS